

Árvores Graduadas e Rubro-Negras

Estruturas de Dados

Prof. Dr. Anderson Bessa da Costa

Universidade Federal de Mato Grosso do Sul

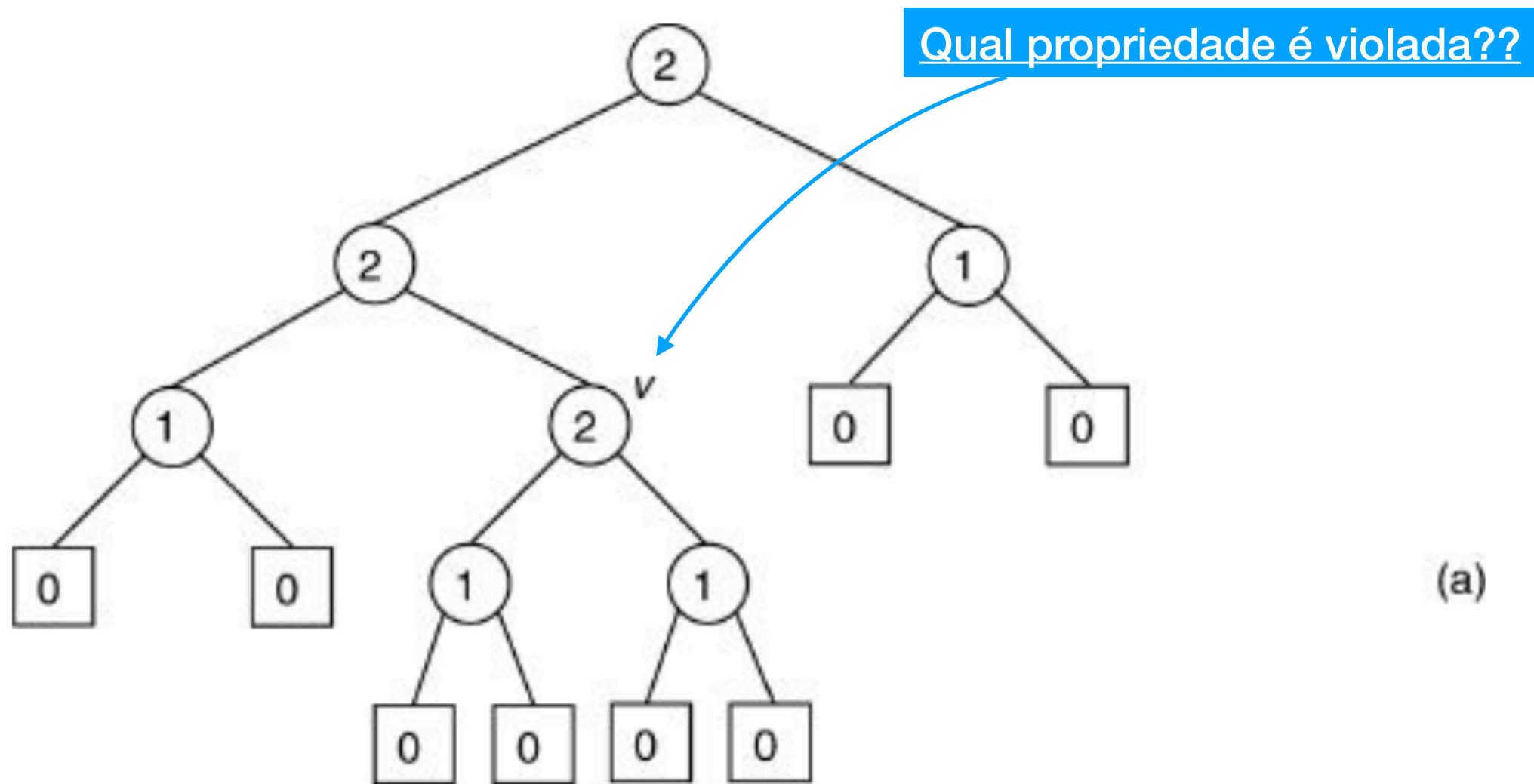
Árvores Graduadas

- Constituem uma generalização das **árvores AVL** e preservam a característica de balanceamento
- Seja T uma BST. Considere a associação de um rótulo inteiro, denominado *posto*, a cada um de seus nós. T é **graduada** quando, para todo $v \in V$, $posto(v)$ satisfaz:
 - (i) Se v é um nó externo, $posto(v) = 0$;
 - (ii) Se v é pai de nó externo, $posto(v) = 1$;
 - (iii) Se v possui pai w ,
 $posto(v) \leq posto(w) \leq posto(v) + 1$;
 - (iv) Se v possui avô w , $posto(v) < posto(w)$.

Árvores Graduadas (Cont.)

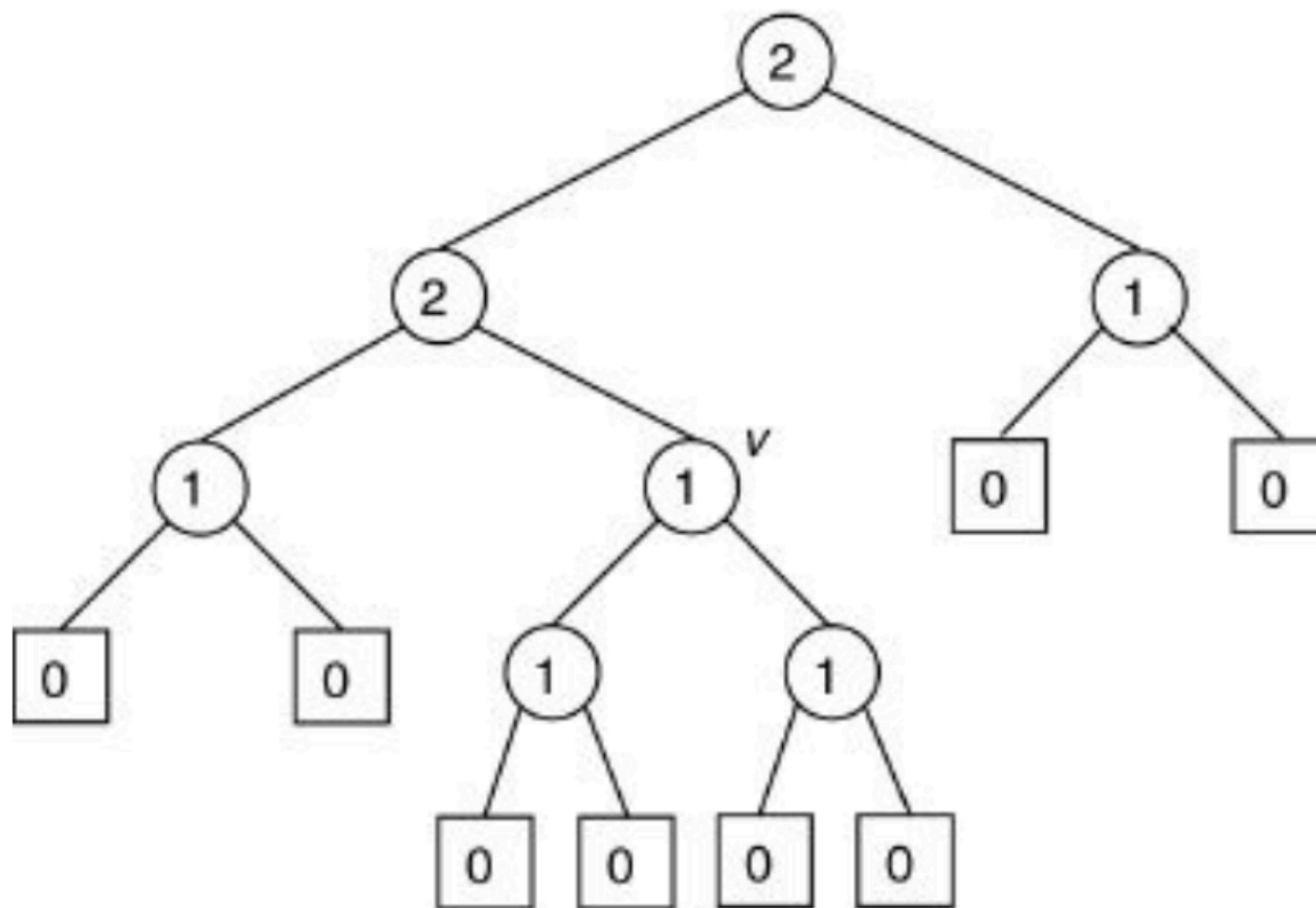
- O nó v é denominado **equilibrado** se satisfaz $(i) - (iv)$
 - E **desequilibrado** em caso contrário
- Naturalmente, uma **árvore graduada** é aquela cujos nós são todos equilibrados

Uma Árvore Não Graduada



Árvore não graduada. Todos os nós estão equilibrados, exceto o nó v .

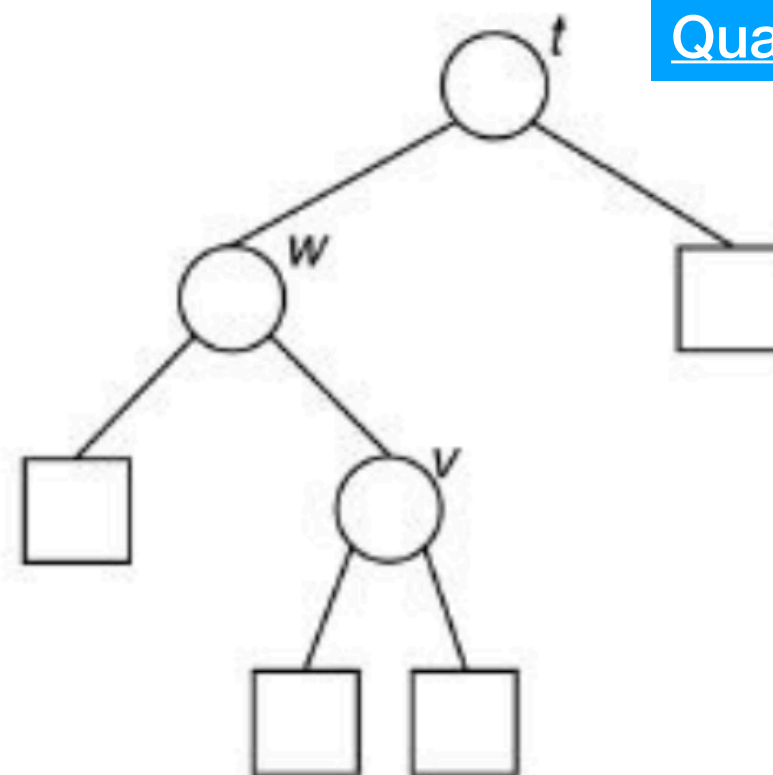
Uma Árvore Graduada



(b)

Árvore graduada. Alterando-se o valor $posto(v)$ para 1, obtém-se a rotulação correta.

Uma Árvore Graduada Impossível



Quais propriedades se contradizem??

(c)

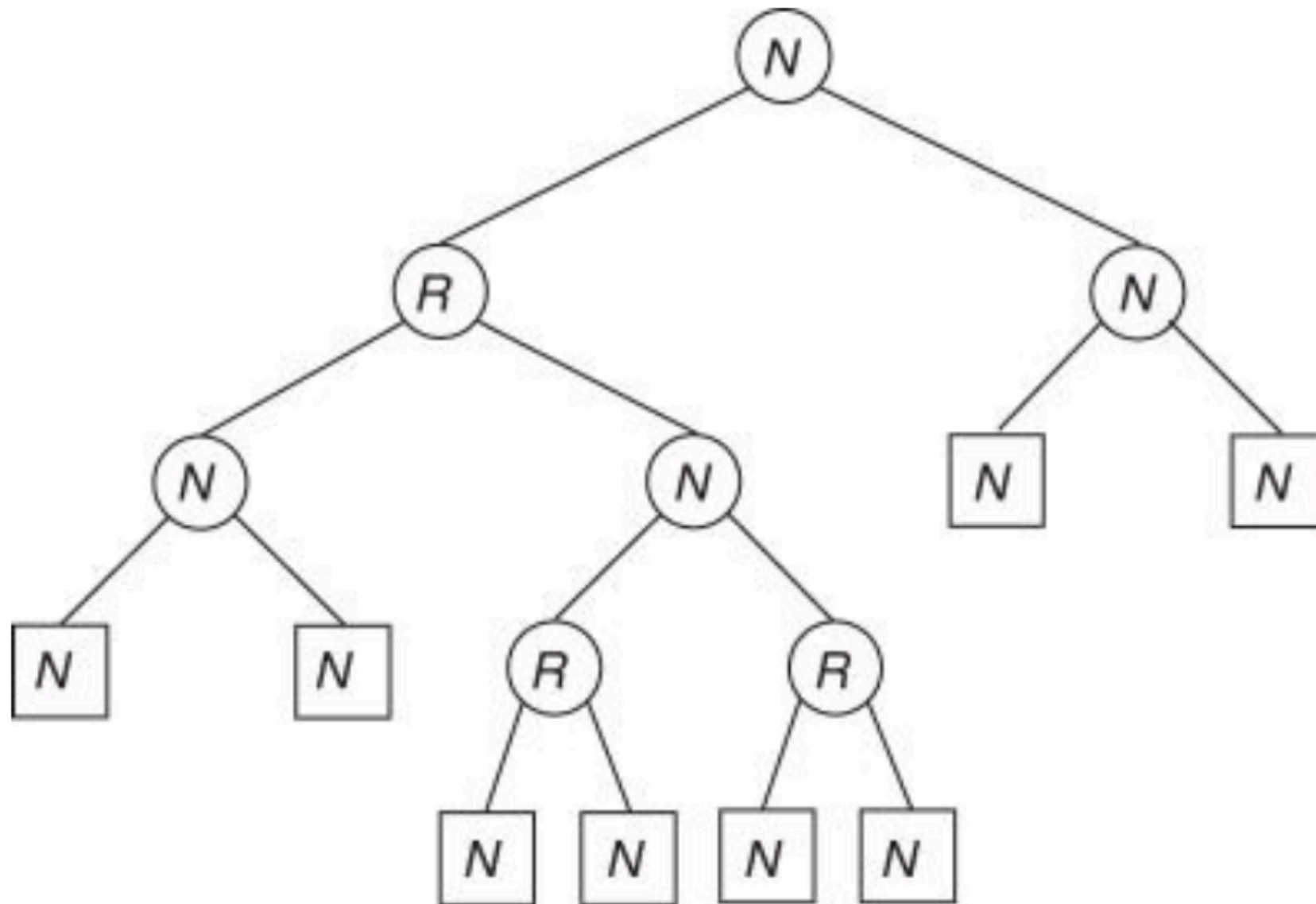
Uma árvore graduada impossível, pois não existe rotulação possível para seus nós que satisfaça (i)-(iv).

Árvores Rubro-negras

Árvores Rubro-negras

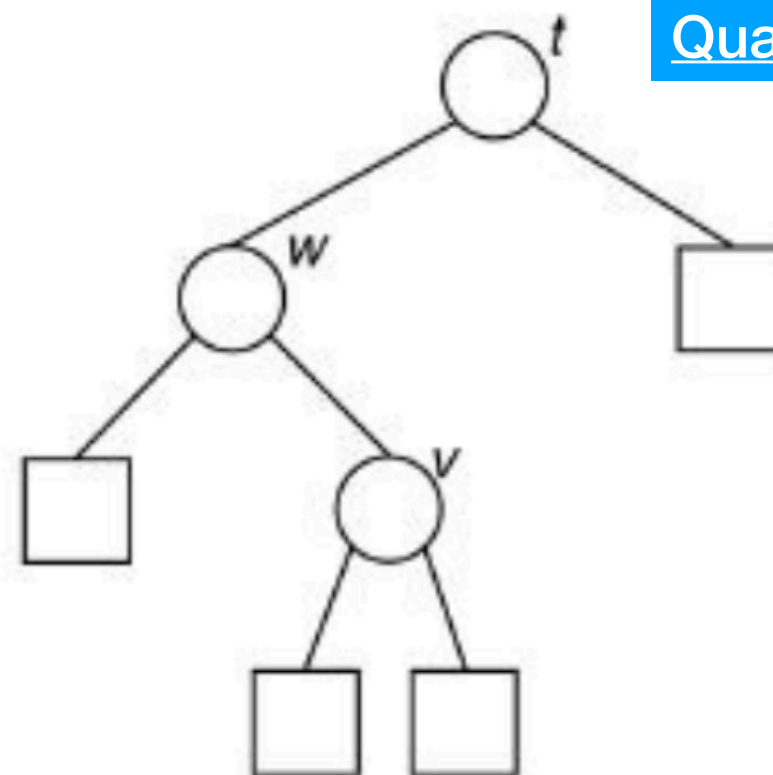
- Equivalente à **árvore graduada**
- Seja T uma árvore binária de busca. Considere agora a associação de uma cor **rubra** ou **negra** a cada um de seus nós. T é **rubro-negra** quando existe uma coloração dos nós de T tal que:
 - (a) se v é externo, v é **negro**;
 - (b) os caminhos de v para seus descendentes nós externos possuem idêntico número de nós **negros**;
 - (c) se v é **rubro** e não é a raiz da árvore então seu pai é **negro**.

Uma Árvore Rubro-negra



Árvore rubro-negra. As cores de seus nós estão representadas pelas iniciais R e N.

Uma Árvore Rubro-negra Impossível



Quais propriedades se contradizem??

(c)

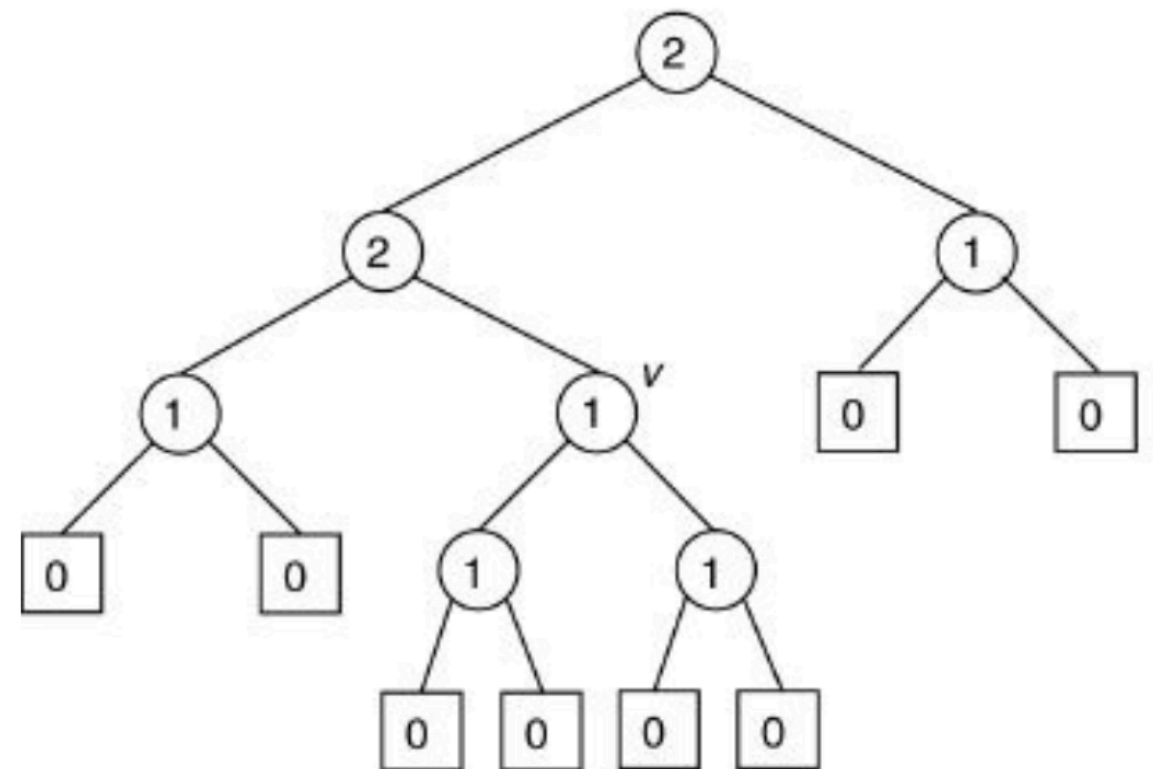
Uma árvore rubro-negra impossível, pois não existe coloração possível para seus nós de modo que (a)-(c) sejam sempre satisfeitos.

Árvore Graduada $\leftrightarrow$ Árvore Rubro-negra

- Existe uma correspondência entre árvores **graduadas** e **rubro-negras**
- Dada uma árvore **graduada**, pode-se colorir seus vértices com as cores **rubra** ou **negra**, de modo que (a) – (c) sejam satisfeitas
- Reciprocamente, em uma árvore **rubro-negra** é possível definir o rótulo $posto(v)$ para cada um de seus vértices, de modo a satisfazer (i) – (iv)

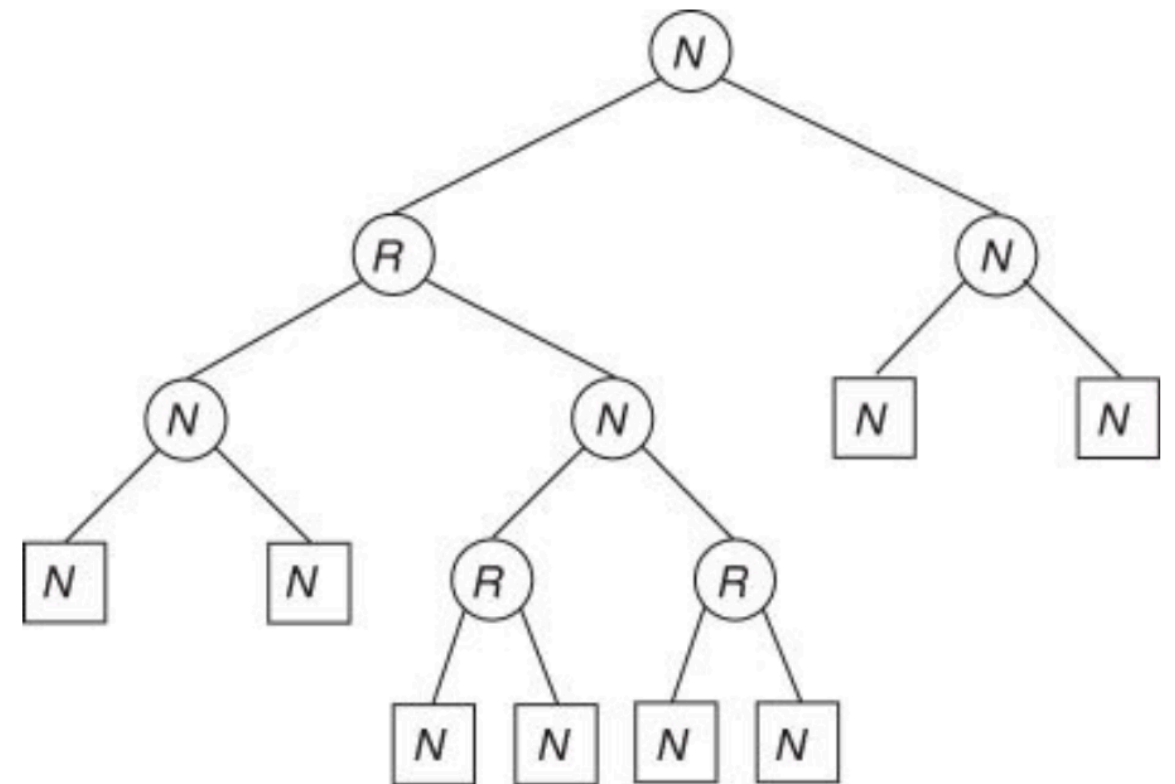
Conversão A: Graduada $\rightarrow$ Rubro-negra

Seja T uma árvore graduada. Atribui-se uma bicoloração a T percorrendo-se a árvore de cima para baixo. Se v é a raiz de T , então $cor(v) := \textit{negra}$. Caso contrário, seja w o pai de v . Define-se $cor(v) := \textit{rubra}$ se $posto(v) = posto(w)$ e $\textit{negra}$, no caso contrário.

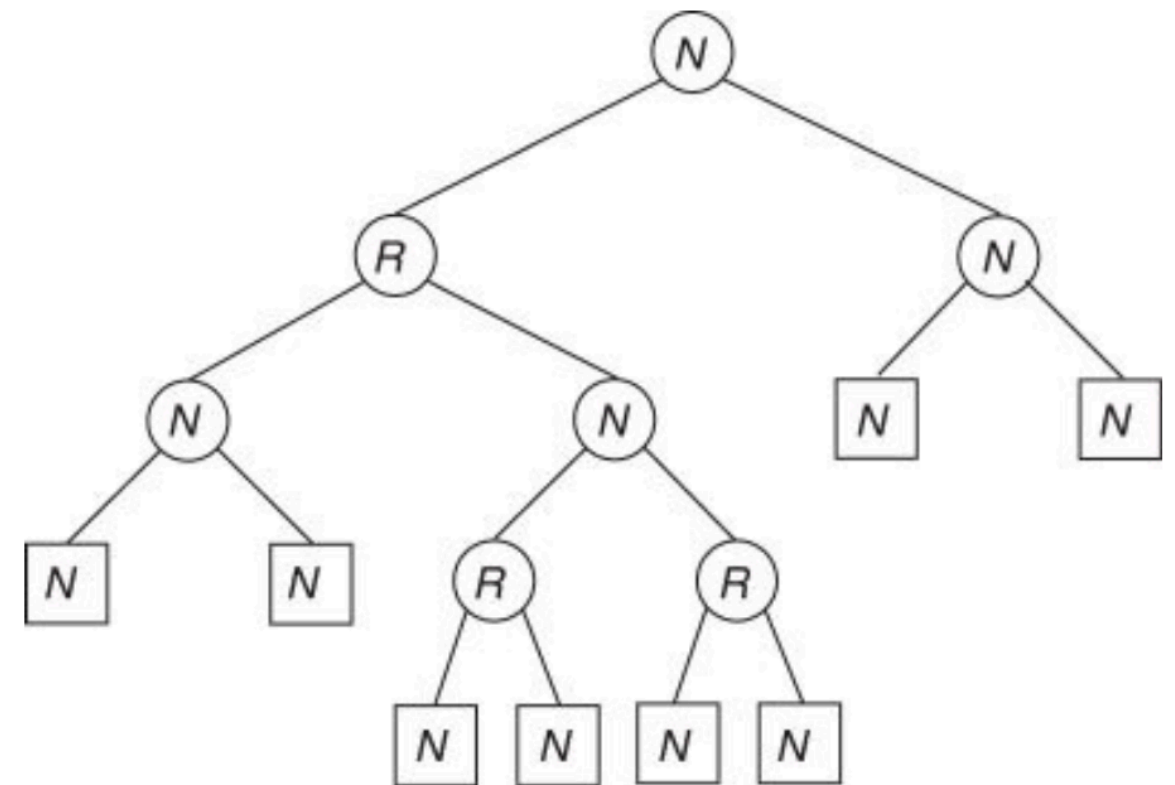
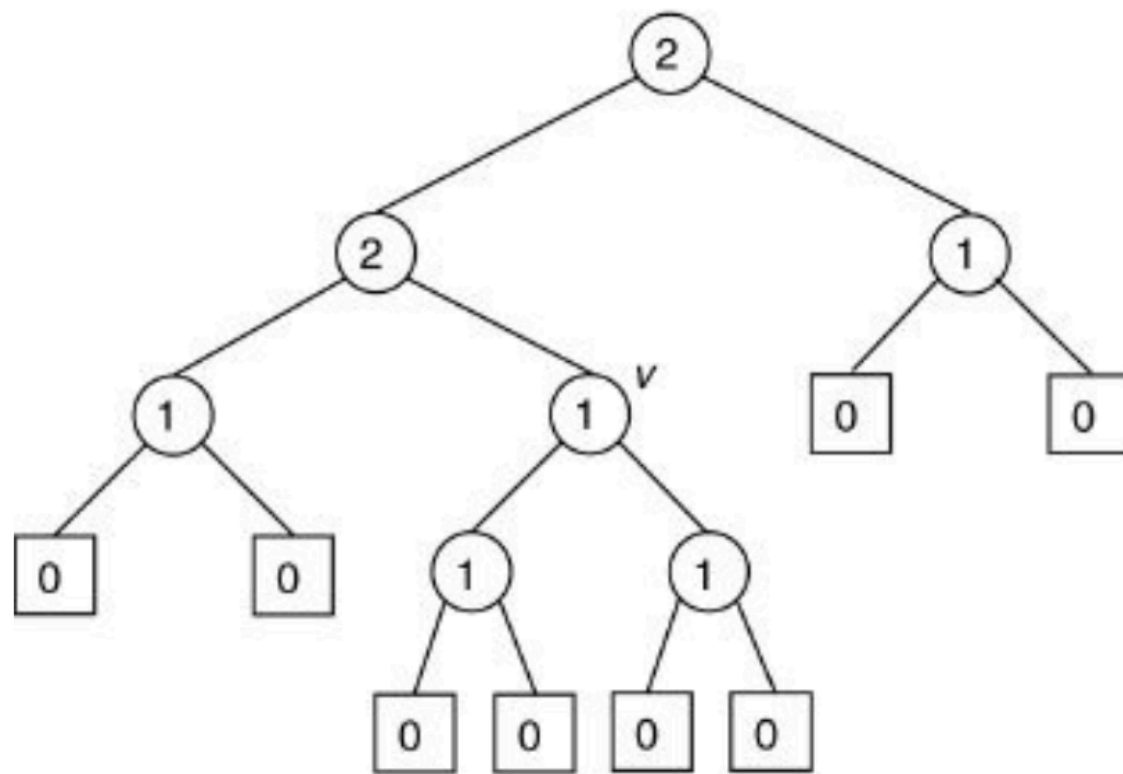


Conversão B: Árvore Rubro-negra → Árvore Graduada

Seja T uma árvore rubro-negra. Efetua-se uma atribuição de valores $posto(v)$ a seus nós percorrendo-se a árvore de baixo para cima. Se v é um nó externo, então $posto(v) := 0$. Caso contrário, seja q um filho (qualquer) de v . Define-se $posto(v) := posto(q)$ se q for *rubro*, ou $posto(v) := posto(q) + 1$ se q for *negro*.



Correspondência Árvores Graduadas $\leftrightarrow$ Rubro-negras



(À esquerda) **Árvore Graduada**; (À direita) **Árvore Rubro-negra**.

Inserção

- A exemplo das árvores AVL, a **inserção** de um novo nó pode **desequilibrar** a árvore
 - Se isto ocorrer, há necessidade do emprego de **operações de equilíbrio** para tornar a árvore novamente graduada
 - Operações em tempo $O(\log n)$

Inserção Árvore Rubro-negra

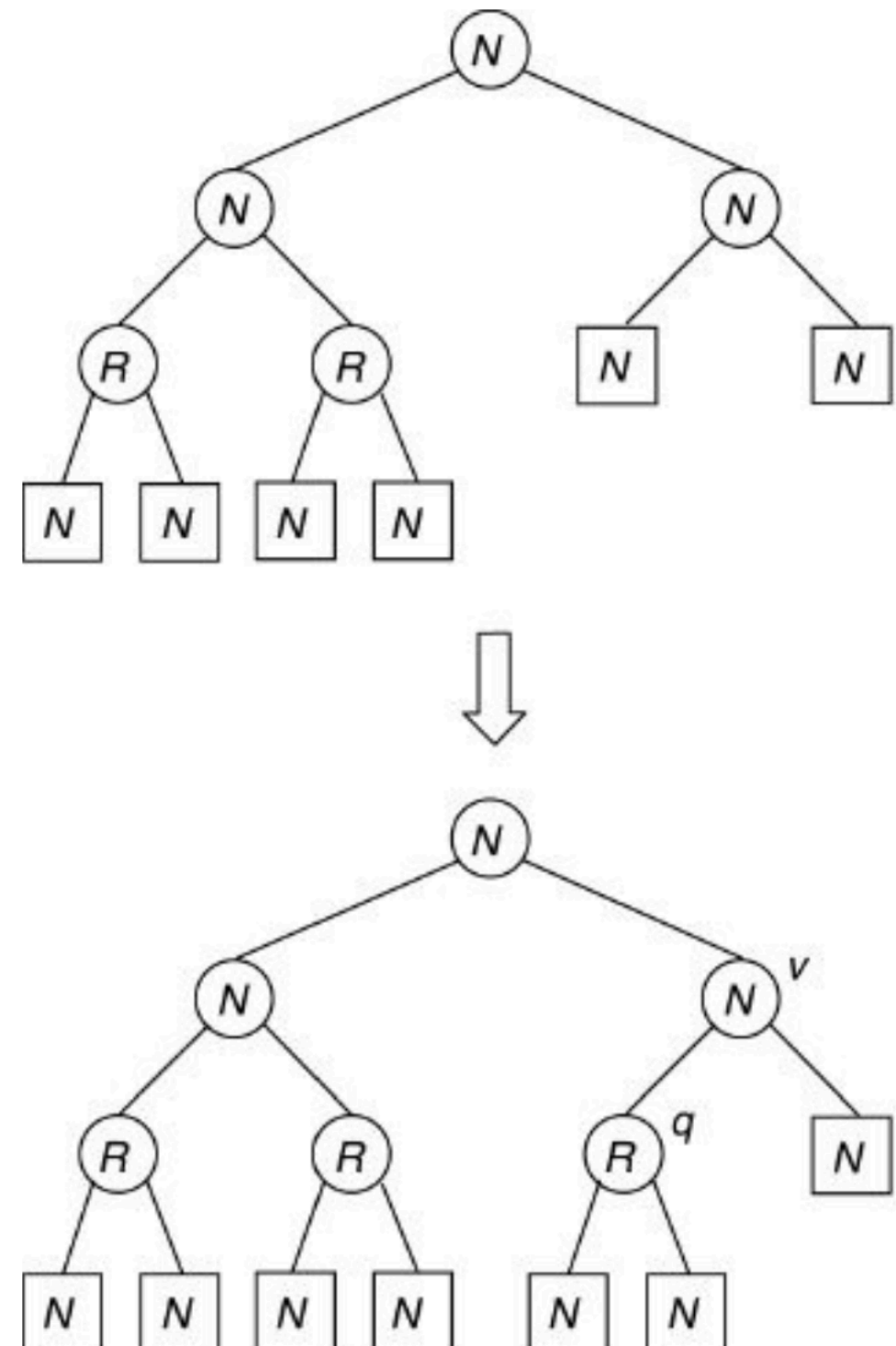
- Seja:
 - $\Rightarrow T$ árvore **rubro-negra**
 - $\Rightarrow q$ um nó a ser incluído
- Suponha que valor de q seja diferente das demais chaves de T
- Nesse caso, o algoritmo de inclusão em árvores binárias substituirá um dos nós externos de T pelo novo nó interno q , obtendo uma nova árvore T^*
- Os filhos esquerdo e direito de q são definidos como nós externos

Inserção Árvore Rubro-negra (Cont.)

- Sejam:
 - $\Rightarrow v$ o pai de q em T^* , se existir
 - $\Rightarrow w$ o avô de q em T^* , se existir
- A questão consiste em verificar se T^* é também **rubro-negra** e, em caso contrário, equilibrar seus nós de forma conveniente
- Para saber se T^* é **rubro-negra**, basta verificar as condições (a) – (c). É imediato que (a) é verdadeiro. Define-se q como rubro. Então (b) também vale. Resta testar (c). As seguintes possibilidades podem ocorrer:

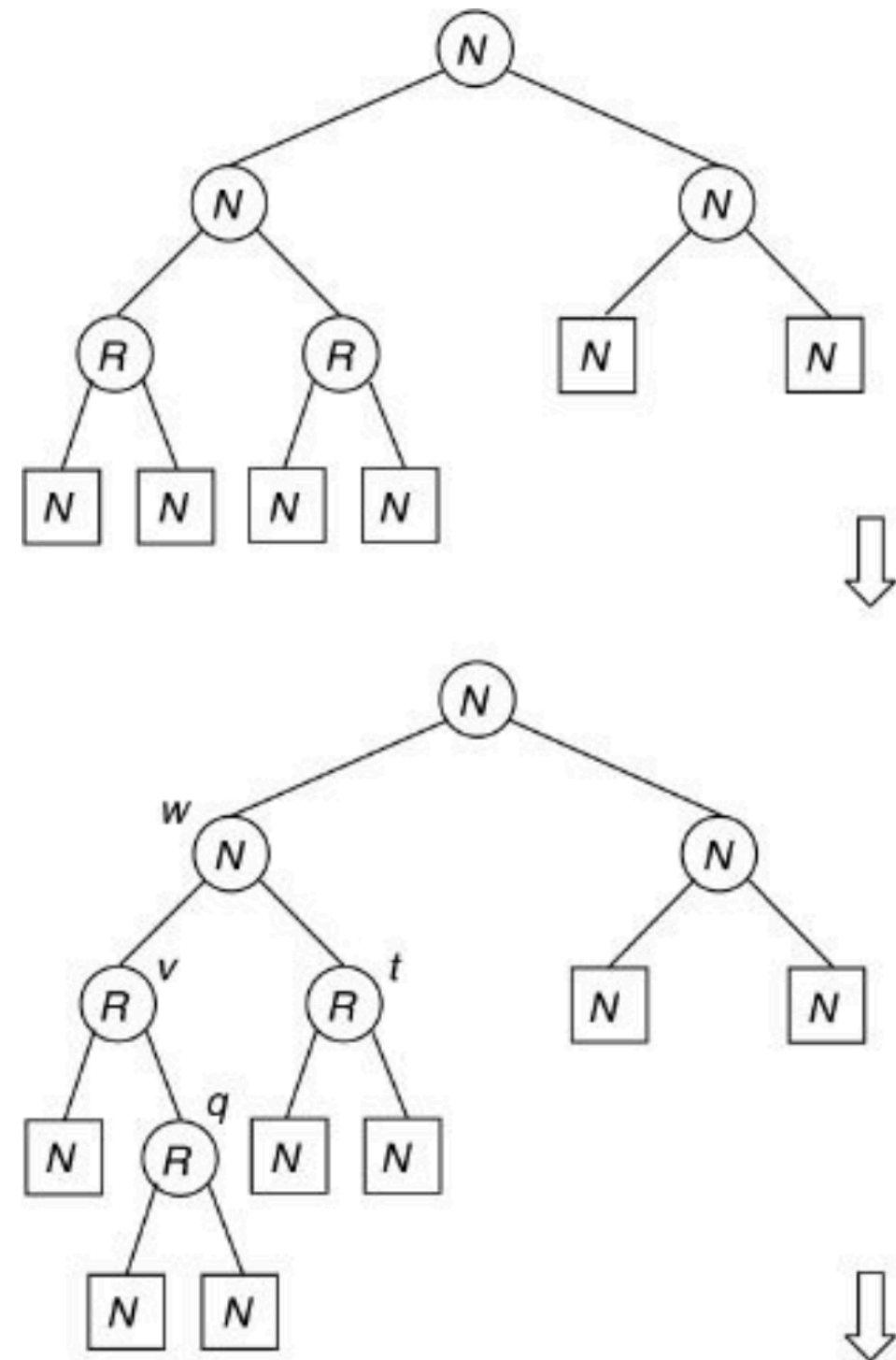
Caso 1: v é negro

- Então (c) é verdadeiro e T^* é **rubro-negra**
- Não há necessidade de operações de equilíbrio



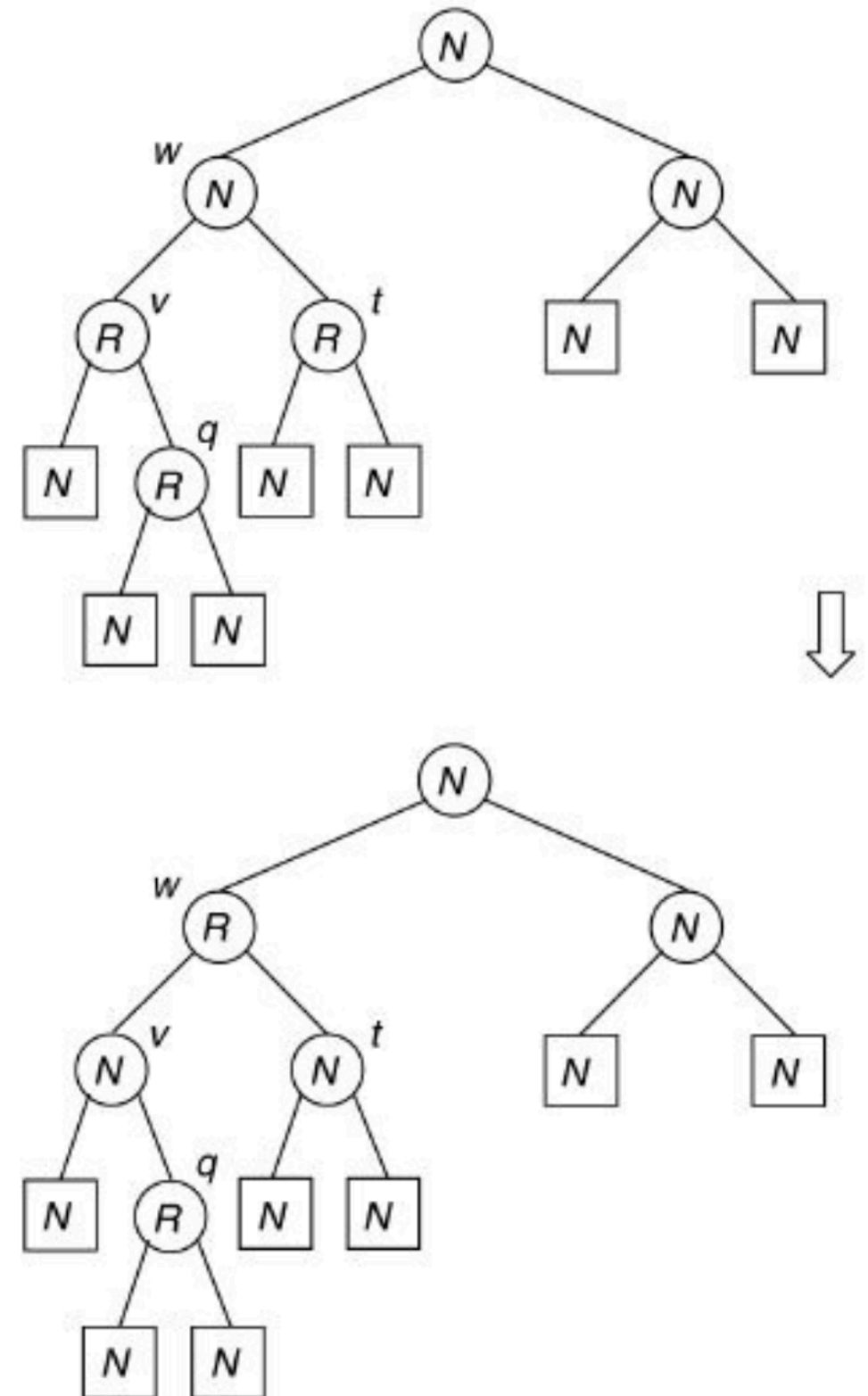
Caso 2: v é rubro

- Nó q não satisfaz (c). Torna-se então necessário equilibrá-lo
- Se v não é a raiz da árvore, o seu pai w é necessariamente negro
- Indaga-se, em seguida, a cor do nó t , irmão de v



Caso 2.1: t é rubro

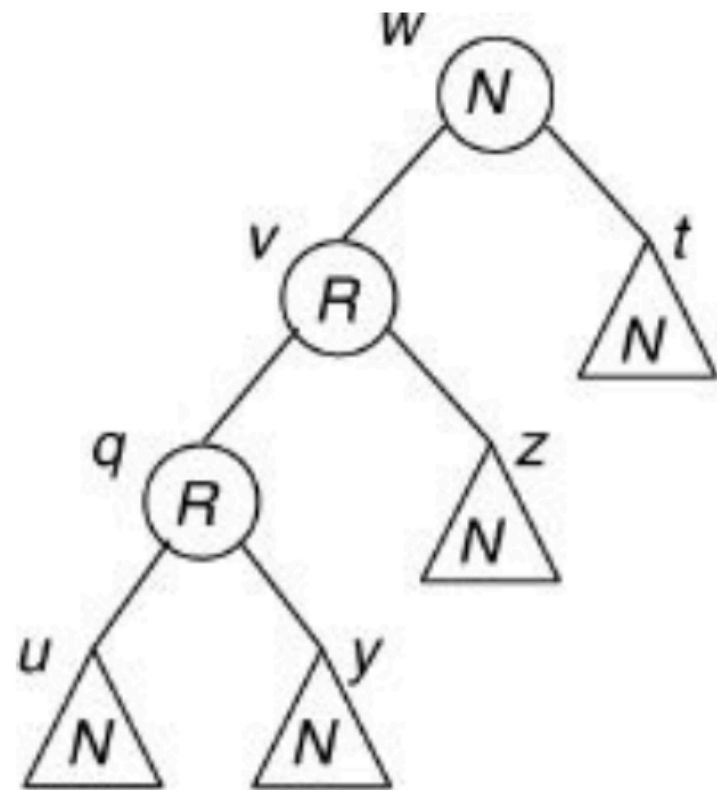
- Altera-se a cor de v , t e w . Os nós v e t tornam-se negros e w , rubro
- Repete-se o processo, com a atribuição $q := w$, redefinindo-se v , w e t como pai, avô e tio de q , respectivamente



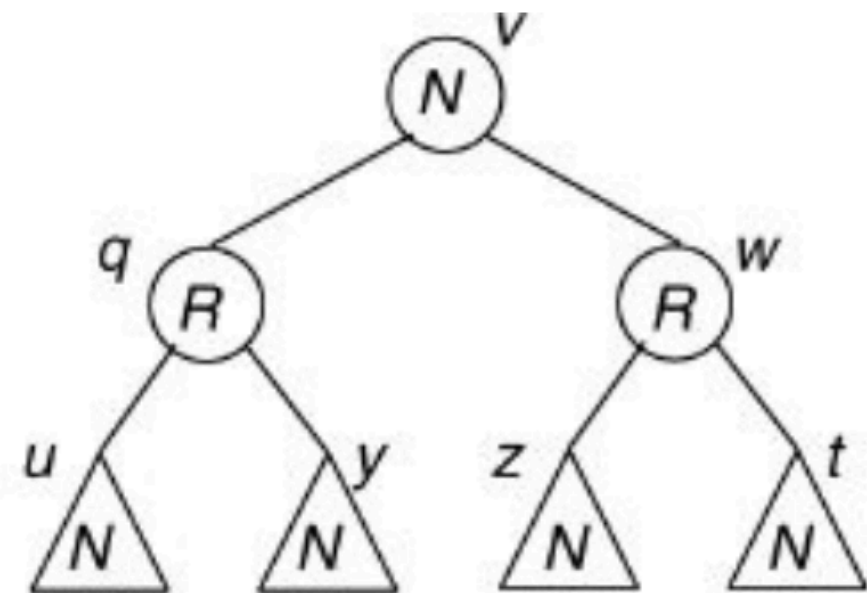
Caso 2.2: t é negro

- Utilizam-se as operações de rotação vistas em AVL
- Basta uma operação de rotação seguida de uma troca de cores conveniente de dois nós para equilibrar q e transformar T^* em árvore rubro-negra

Caso 2.2.1: q é filho esquerdo de v
e v é filho esquerdo de w

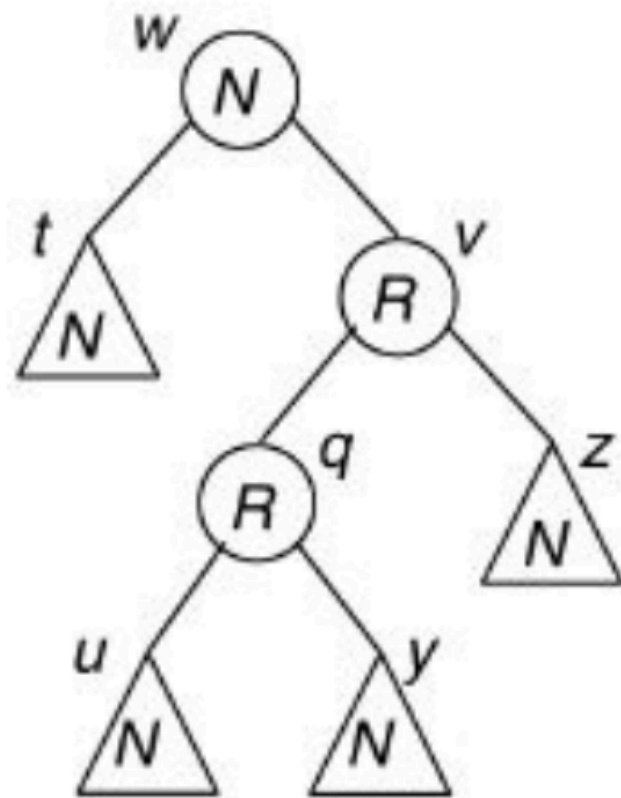


→
rotação
direita

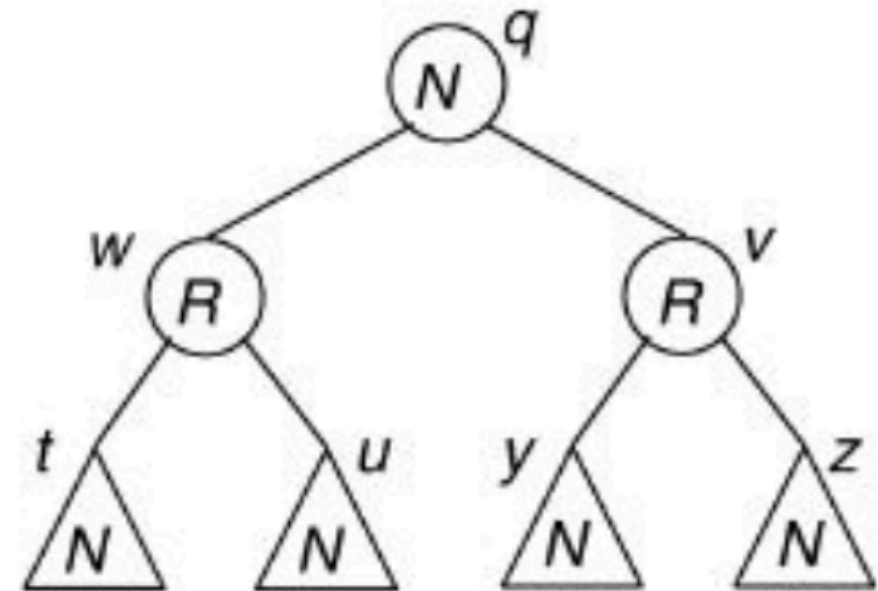


(a)

Caso 2.2.2: q é filho esquerdo de v
e v é filho direito de w

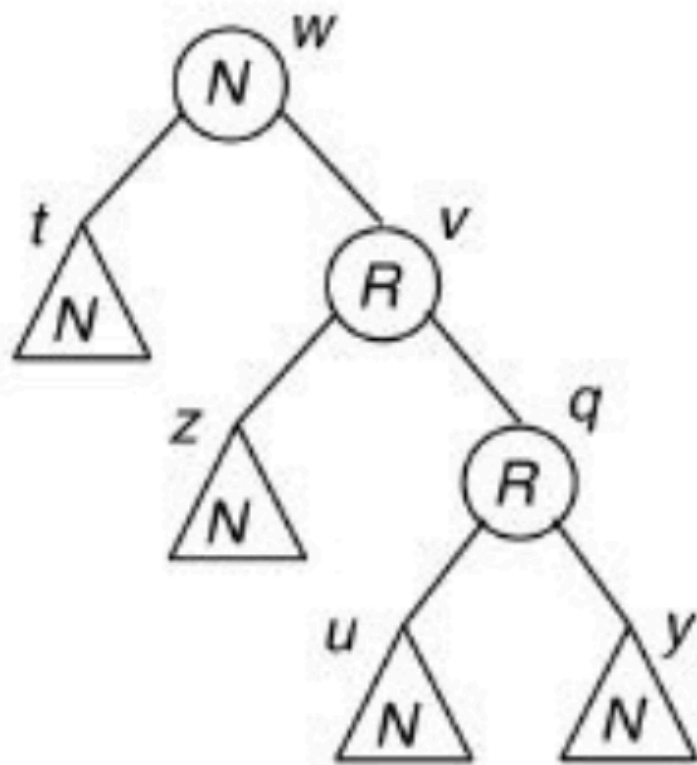



 rotação
 dupla
 esquerda

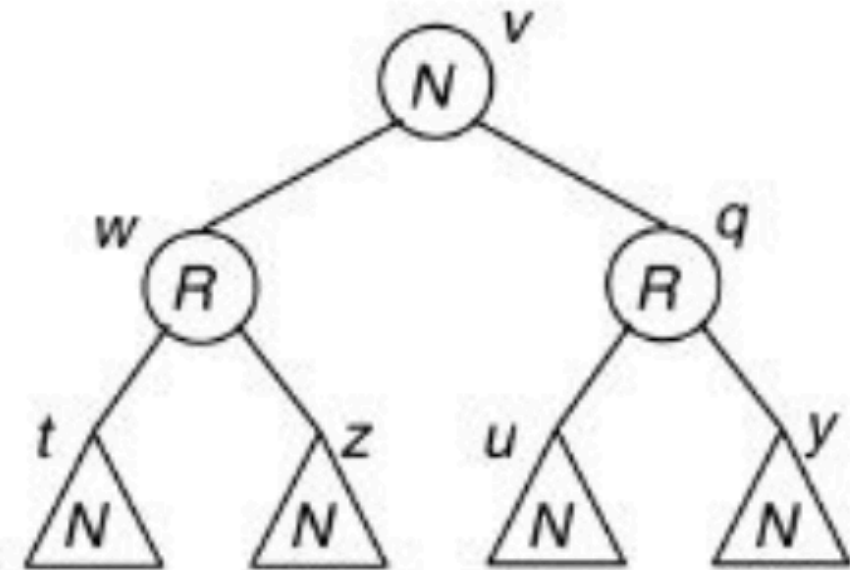


(b)

Caso 2.2.3: q é filho direito de v e v é filho direito de w

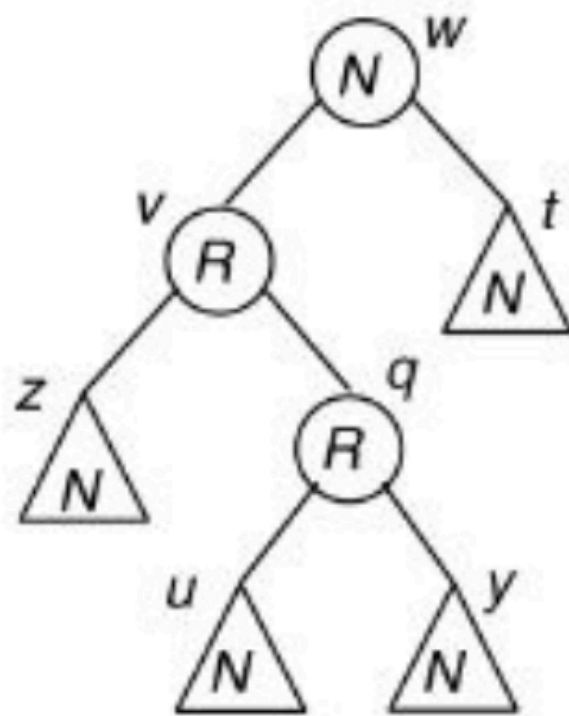


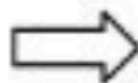
→
rotação
esquerda

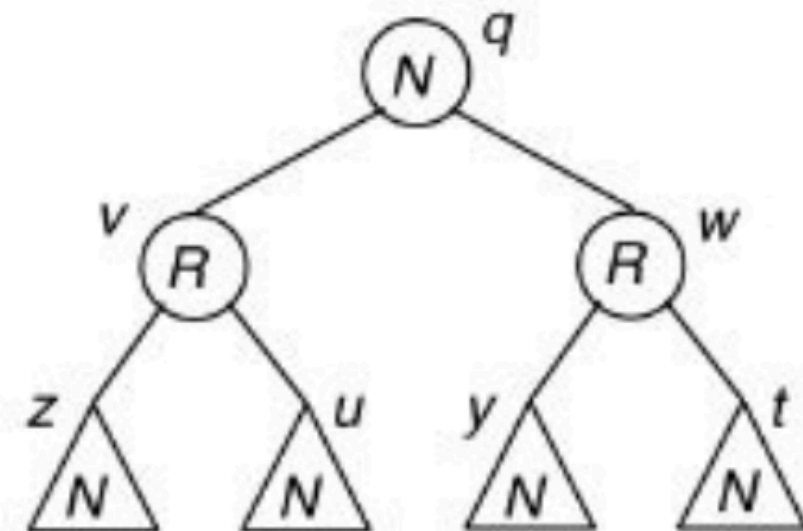


(c)

Caso 2.2.4: q é filho direito de v e v é filho esquerdo de w




 rotação
 dupla
 direita



(d)

Ao Final, T^* é Rubro-negra

- Observe que o Caso 2.1 pode ocorrer iterativamente
 - $O(\log n)$ vezes, uma vez que o desequilíbrio pode ser propagado de um nó para o seu avô
- Demais casos somente podem ocorrer, no máximo, uma vez cada

Inclusão em Árvore Rubro-negra

- O procedimento $ins - RN(x, p_{tv}, p_{tw}, p_{tr}, a)$ efetua uma busca para localizar a posição na árvore onde o novo nó de chave x será incluído
- Os ponteiros p_{tv} , p_{tw} , p_{tr} apontam para o nó corrente da busca, seu pai e seu avô, respectivamente
- A chamada externa é $ins - RN(x, p_{tv}, \lambda, \lambda, 1)$, com $p_{tv} = p_{traiz}$
- O algoritmo considera então a raiz da árvore construída sempre como negra

procedimento *ins- RN* ($x, p_{tv}, p_{tw}, p_{tr}, a$)

se $p_{tv} = \text{externo}$ **então**

ocupar(p_{tv})

$p_{tv} \uparrow. \text{esq} := p_{tv} \uparrow. \text{dir} := \text{externo}$

$p_{tv} \uparrow. \text{chave} := x; p_{tv} \uparrow. \text{cor} := R$

se $p_{traiz} = \text{externo}$ **então**

$p_{tv} \uparrow. \text{cor} := N$ % novo nó é raiz

$p_{traiz} := p_{tv}$

senão se $x < p_{tw} \uparrow. \text{chave}$ **então** % incluir nó na árvore

$p_{tw} \uparrow. \text{esq} := p_{tv}$

senão $p_{tw} \uparrow. \text{dir} := p_{tv}$

senão se $x \neq p_{tv} \uparrow. \text{chave}$ **então**

se $x < p_{tv} \uparrow. \text{chave}$ **então**

$p_{tq} := p_{tv} \uparrow. \text{esq}$

senão $p_{tq} := p_{tv} \uparrow. \text{dir}$

ins- RN ($x, p_{tq}, p_{tv}, p_{tw}, a$)

procedimento *ins- RN* ($x, p_{tv}, p_{tw}, p_{tr}, a$)

se $p_{tv} = \text{externo}$ **então**

ocupar(p_{tv})

$p_{tv} \uparrow. \text{esq} := p_{tv} \uparrow. \text{dir} := \text{externo}$

$p_{tv} \uparrow. \text{chave} := x; p_{tv} \uparrow. \text{cor} := R$

se $p_{traiz} = \text{externo}$ **então**

$p_{tv} \uparrow. \text{cor} := N$ % novo nó é raiz

$p_{traiz} := p_{tv}$

senão se $x < p_{tw} \uparrow. \text{chave}$ **então** % incluir nó na árvore

$p_{tw} \uparrow. \text{esq} := p_{tv}$

senão $p_{tw} \uparrow. \text{dir} := p_{tv}$

senão se $x \neq p_{tv} \uparrow. \text{chave}$ **então**

se $x < p_{tv} \uparrow. \text{chave}$ **então**

$p_{tq} := p_{tv} \uparrow. \text{esq}$

senão $p_{tq} := p_{tv} \uparrow. \text{dir}$

ins- RN ($x, p_{tq}, p_{tv}, p_{tw}, a$)

se $a = 1$ **então** *rota*($p_{tq}, p_{tv}, p_{tw}, p_{tr}, a$)

senão se $a = 0$ **então** $a := 1$

senão “inserção inválida”

procedimento *rota*($p_{tq}, p_{tv}, p_{tw}, p_{tr}, a$)

$a := 2$

se $p_{tv} \uparrow. \text{cor} = R$ **então**

se $p_{tv} = p_{tw} \uparrow. \text{esq}$ **então**

$p_{tt} := p_{tw} \uparrow. \text{dir}$

senão $p_{tt} := p_{tw} \uparrow. \text{esq}$

se $p_{tt} \uparrow. \text{cor} = R$ **então** % Caso 2.1

$p_{tv} \uparrow. \text{cor} := N; p_{tw} \uparrow. \text{cor} := R$

```

senão  $ptw \uparrow. dir := ptv$ 
senão se  $x \neq ptv \uparrow. chave$  então
    se  $x < ptv \uparrow. chave$  então
         $ptq := ptv \uparrow. esq$ 
    senão  $ptq := ptv \uparrow. dir$ 
     $ins- RN(x, ptq, ptv, ptw, a)$ 
    se  $a = 1$  então  $rota(ptq, ptv, ptw, ptr, a)$ 
    senão se  $a = 0$  então  $a := 1$ 
senão “inserção inválida”
procedimento  $rota(ptq, ptv, ptw, ptr, a)$ 
     $a := 2$ 
    se  $ptv \uparrow. cor = R$  então
        se  $ptv = ptw \uparrow. esq$  então
             $ptt := ptw \uparrow. dir$ 
        senão  $ptt := ptw \uparrow. esq$ 
        se  $ptt \uparrow. cor = R$  então % Caso 2.1
             $ptv \uparrow. cor := N; \quad ptw \uparrow. cor := R$ 
             $ptt \uparrow. cor := N$ 
             $a := 0$ 
        senão  $ptw \uparrow. cor := R$  % Caso 2.2
            se  $ptq = ptv \uparrow. esq$  então
                se  $ptv = ptw \uparrow. esq$  então
                     $aux := ptv; \quad ptv \uparrow. cor := N$  % Caso 2.2.1
                     $ptw \uparrow. esq := ptv \uparrow. Dir$ 
                     $ptv \uparrow. dir := ptw$ 
                senão  $aux := ptq; \quad ptq \uparrow. cor := N$  % Caso 2.2.2
                     $ptw \uparrow. dir := ptq \uparrow. esq$ 
                     $ptv \uparrow. esq := ptq \uparrow. dir$ 

```

```

se  $ptv \uparrow. cor = R$  então
    se  $ptv = ptw \uparrow. esq$  então
         $ptt := ptw \uparrow. dir$ 
    senão  $ptt := ptw \uparrow. esq$ 
    se  $ptt \uparrow. cor = R$  então                                % Caso 2.1
         $ptv \uparrow. cor := N; \quad ptw \uparrow. cor := R$ 
         $ptt \uparrow. cor := N$ 
         $a := 0$ 
    senão  $ptw \uparrow. cor := R$                                 % Caso 2.2
        se  $ptq = ptv \uparrow. esq$  então
            se  $ptv = ptw \uparrow. esq$  então
                 $aux := ptv; \quad ptv \uparrow. cor := N$                                 % Caso 2.2.1
                 $ptw \uparrow. esq := ptv \uparrow. Dir$ 
                 $ptv \uparrow. dir := ptw$ 
            senão  $aux := ptq; \quad ptq \uparrow. cor := N$                                 % Caso 2.2.2
                 $ptw \uparrow. dir := ptq \uparrow. esq$ 
                 $ptv \uparrow. esq := ptq \uparrow. dir$ 
                 $ptq \uparrow. esq := ptv; \quad ptq \uparrow. dir := ptv$ 
        senão se  $ptv = ptw \uparrow. dir$  então                                % caso 2.2.3
             $aux := ptv; \quad ptv \uparrow. cor := N$ 
             $ptw \uparrow. dir := ptv \uparrow. esq$ 
             $ptv \uparrow. esq := ptw$ 
        senão  $aux := ptq; \quad ptq \uparrow. cor := N$                                 % caso 2.2.4
             $ptw \uparrow. esq := ptq \uparrow. dir$ 
             $ptv \uparrow. dir := ptq \uparrow. esq$ 
             $ptq \uparrow. esq := ptv; \quad ptq \uparrow. dir := ptw$ 
    se  $ptr \neq \lambda$  então
        se  $ptq \uparrow. chave < ptr \uparrow. chave$  então

```


% Caso 2.2.1

$aux := p_{tv}; \quad p_{tv} \uparrow. cor := N$
 $p_{tw} \uparrow. esq := p_{tv} \uparrow. Dir$
 $p_{tv} \uparrow. dir := p_{tw}$

% Caso 2.2.2

senão $aux := p_{tq}; \quad p_{tq} \uparrow. cor := N$
 $p_{tw} \uparrow. dir := p_{tq} \uparrow. esq$
 $p_{tv} \uparrow. esq := p_{tq} \uparrow. dir$
 $p_{tq} \uparrow. esq := p_{tw}; \quad p_{tq} \uparrow. dir := p_{tv}$

% caso 2.2.3

senão se $p_{tv} = p_{tw} \uparrow. dir$ **então**
 $aux := p_{tv}; \quad p_{tv} \uparrow. cor := N$
 $p_{tw} \uparrow. dir := p_{tv} \uparrow. esq$
 $p_{tv} \uparrow. esq := p_{tw}$

% caso 2.2.4

senão $aux := p_{tq}; \quad p_{tq} \uparrow. cor := N$
 $p_{tw} \uparrow. esq := p_{tq} \uparrow. dir$
 $p_{tv} \uparrow. dir := p_{tq} \uparrow. esq$
 $p_{tq} \uparrow. esq := p_{tv}; \quad p_{tq} \uparrow. dir := p_{tw}$

se $ptr \neq \lambda$ **então**
 se $p_{tq} \uparrow. chave < ptr \uparrow. chave$ **então**
 $ptr \uparrow. esq := aux$
 senão $ptr \uparrow. dir := aux$
senão $p_{traiz} := aux$

$p_{traiz} \uparrow. cor := N$

Complexidade Inclusão

- Cada chamada consome tempo constante
- Portanto, inclusão e equilíbrio possuem tempo $O(\log n)$
- Espaço adicional necessário: um bit por nó para informar a cor desse nó

Remoção em Árvores Rubro-negras

- Estratégia segue os processos anteriores
- Após a operação de exclusão propriamente dita, verifica-se se algum nó tornou-se desequilibrado
- Em caso positivo, efetua-se o equilíbrio da árvore
- Tempo $O(\log n)$

Referências

- SZWARCFITER, Jayme Luiz; MARKENZON, Lilian.
Estruturas de Dados e seus Algoritmos. Edição: 3a. Editora:
LTC. 2010